

CFD Project 1

October 15, 2025

Eleni Kambouris

Abstract -

This report examines the effects of mesh grid size on the accuracy of the Laplace Scheme of the 2D Heat Equation on a uniform unit plate. The accuracy and convergence rates are tested for 4x4, 8x8, 16x16, and 32x32 grids. These results are compared to exact solutions, truncation error, and an improved scheme.

Introduction –

Sometimes it is difficult or resource consuming to directly solve a PDE for a physical system. In cases like these, we choose to discretize the system and solve a numerical scheme. Ideally, the problem is well-posed, and the scheme chosen is a good representation of our PDE. For example, take the 2D Heat Equation over a uniform, unit plate.

$$\nabla^2 T = 0$$

This can be discretized into a form that satisfies Laplace's Equation

$$T_{j,k}^{n+1} = \frac{1}{4} (T_{j-1,k}^n + T_{j+1,k}^n + T_{j,k+1}^n + T_{j,k-1}^n)$$

where j, k label the points of a uniform ($\Delta x = \Delta y = h$) square grid. This is only one scheme that can be applied. We also apply several initial and boundary conditions. We take

$$T(0, y) = 0,$$

$$T(x, 0) = 0,$$

$$T(1, y) = \sin(\pi y),$$

$$T(x, 1) = \sin(\pi x)$$

and all other initial temperatures are assumed to be 0.

We can try to come up with a better scheme to replace just averaging the 4-surrounding points to calculate the temperature at a given “node.” Take for example, a scheme that looks at diagonal points from the node as well, yielding a total of 8 points being looked at and 9 points in total.

$$T_{j,k}^{n+1} = \frac{1}{20} [4(T_{j-1,k}^n + T_{j+1,k}^n + T_{j,k+1}^n + T_{j,k-1}^n) + (T_{j+1,k+1}^n + T_{j+1,k-1}^n + T_{j-1,k+1}^n + T_{j-1,k-1}^n)]$$

This scheme is better because it is taking more points into account that would affect the temperature in the real world. Our 5-point scheme has a much slower implementation of how surrounding points will affect the 4 we are claiming have influence. This way, we get much closer to looking at all the points at once. Ideally, this will be more accurate and hopefully require fewer iterations to converge.

Derivations/Methods –

To test the accuracy of our results, it is good to compare with other forms of error or solution types. We derive an expression for truncation error, the exact solution, and confirm that the proposed scheme is fact consistent on a uniform grid. The derivations are seen in Appendix B. To solve the scheme, we use MATLAB to iterate through the scheme until the convergence threshold, 10^{-5} , is met. This convergence criteria in compared to the L_∞ norm as a condition of continuing to iterate or stopping. Additionally, MATLAB is used to generate all plots, compare order of error to order of truncation error, and compare the exact and numerical solutions. The code can be seen in Appendix A.

Results/Discussion –

The larger the grid size, the smoother the contour plot is. If we were to zoom in, the lines would still look rather linear, but there is a visible difference in each step up in grid size.

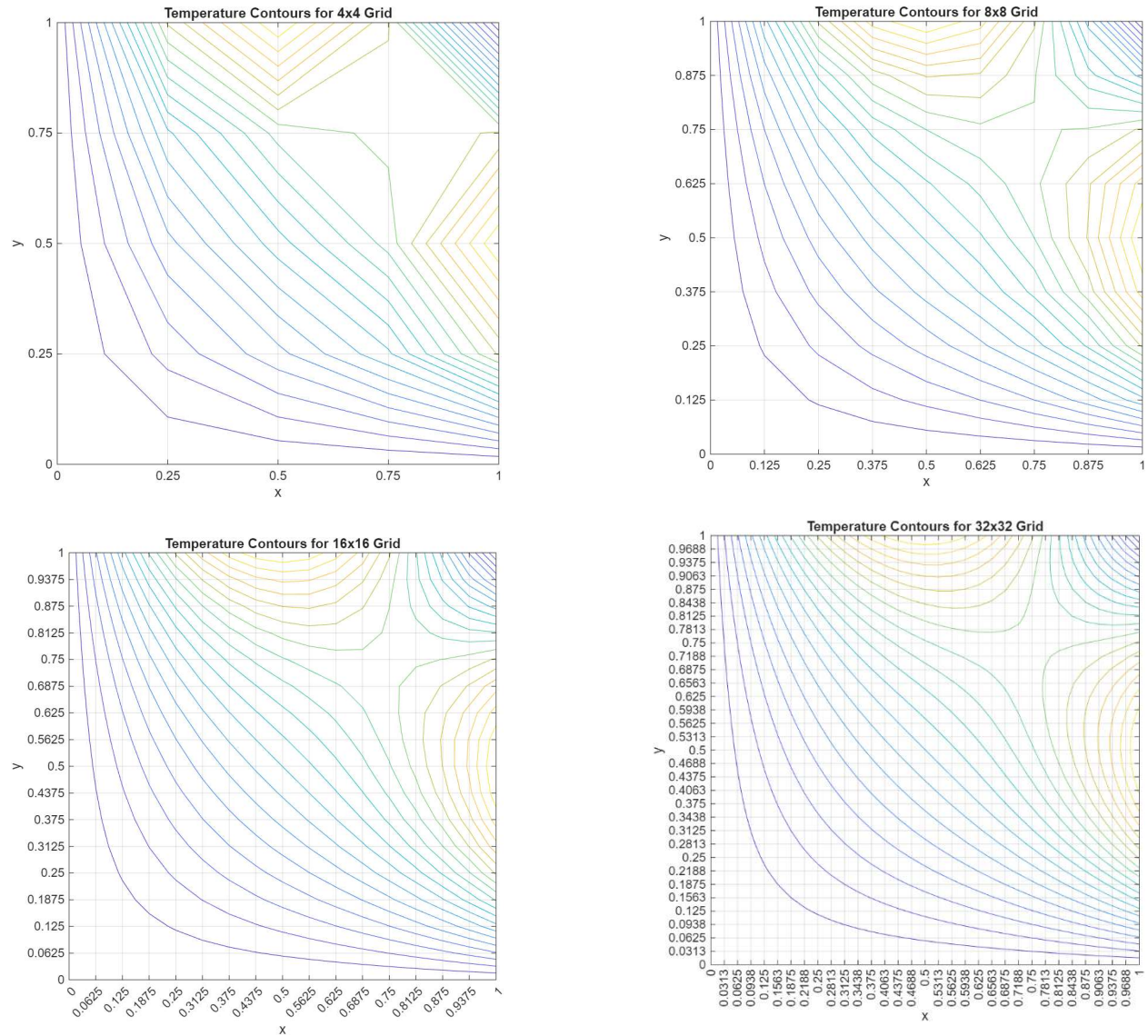


Figure 1. Contour Plots for all grid sizes of the 5-point Scheme.

This is shown clearly in Fig. 1 above. Grid lines are added to depict the intersections, or nodes, where temperature calculations would occur for each simulation. This helps highlight the increase in smoothness of the contour plot when the grid size is expanded. We see that the 32x32 plot looks much smoother than the 4x4 plot, this is as expected because the temperature is solved for in many more locations, so it is going to be a better approximation of the exact solution. We are refining the mesh.

To ensure that the numerical scheme is actually converging, we take the infinity-norm, which is essentially the maximum error, so we know that all other errors must be lower, and we should be below our desired convergence criteria. To observe the behavior we plot the convergence history ($\log_{10} \|T^{n+1} - T^n\|_{\infty}$) vs. iteration number. The plots exhibit very similar behavior for all grid sizes and it can be seen that the solutions have in fact converged. This is seen in Fig. 2 below.

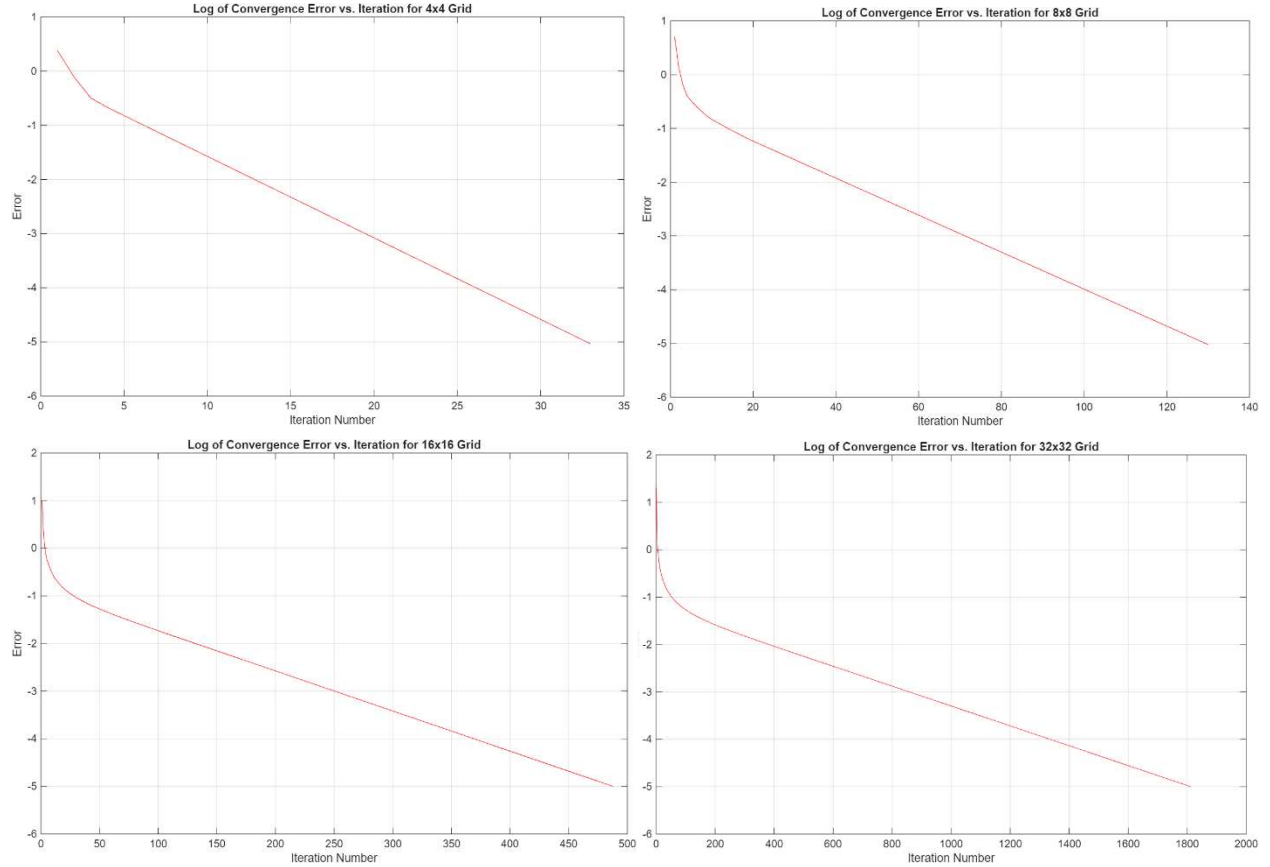


Figure. 2. Convergence history of each grid size.

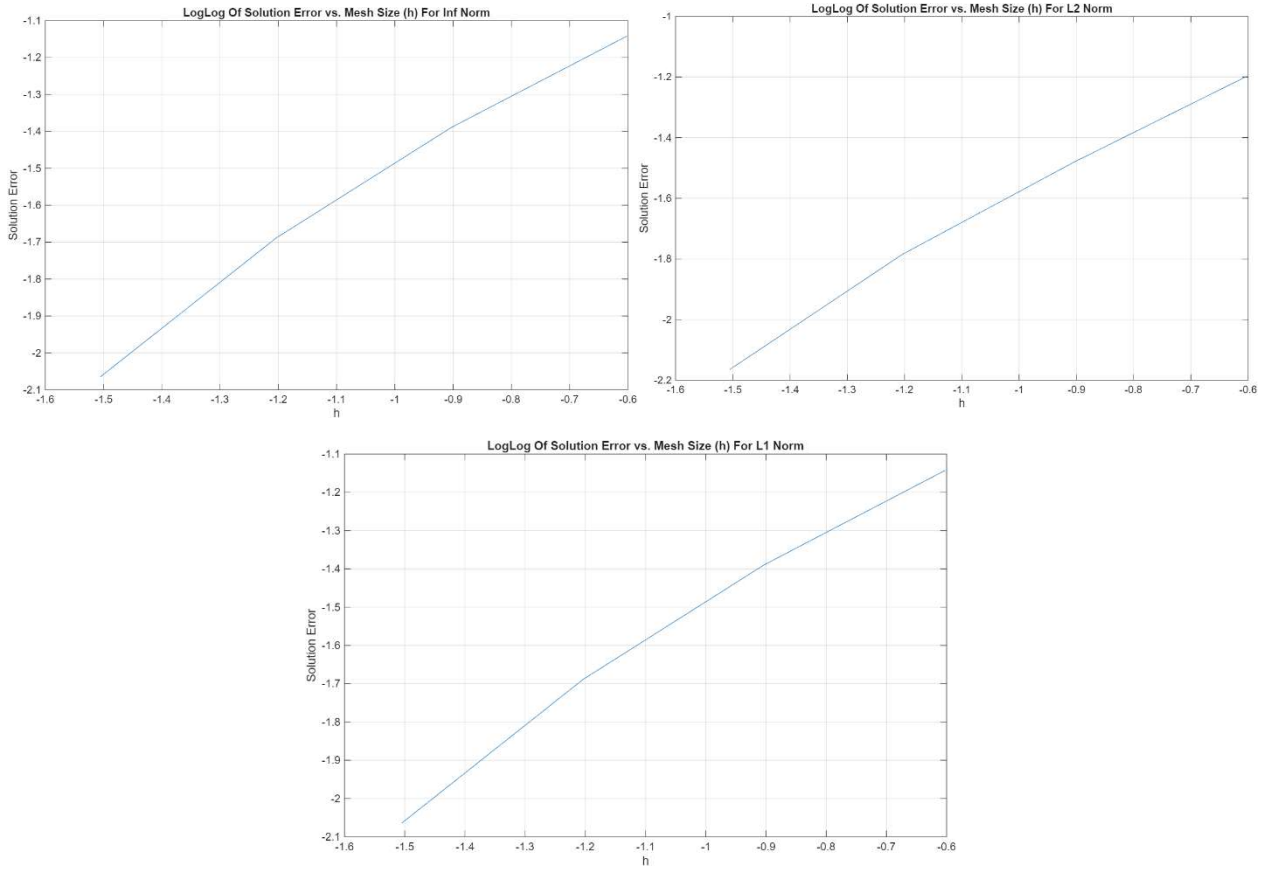
We can also see that the smaller grids require fewer iterations. This makes sense as there are not as many points to iterate over repeatedly. The continuously decreasing behavior until order -5 indicates that this scheme is converging. The infinity norm is a good choice because it is the maximum error, so all other errors must be lower, and we are sure to meet our criteria with a scheme that has converged.

Next, we compare the error in the converged numerical solution with the exact solution. This is done by taking each type of norm for each grid size. The findings are summarized below in Table 1. We see that the error decreases as the grid size increases. This is the expected behavior because our discretized solutions should be getting closer the more points we evaluate it at.

Table. 1. Error between Converged and Exact Solutions.

	4x4	8x8	16x16	32x32
L _{inf}	0.0721	0.0408	0.0205	0.0086
L2	0.0635	0.0331	0.0164	0.0069
L1	0.0721	0.0408	0.0205	0.0086

In the derivations we find the truncation error to be second-order accurate, when we look at how the error scales with grid size, we should see this h^2 behavior. A good way to check this behavior would be taking a loglog plot of the mesh size and the solution error. The slope, p , will correspond with h^p so we expect to see the plots with a slope of 2. Figure 3 depicts these plots for each norm type.

**Figure. 3.** Loglog plots of solution error vs. mesh size.

When the slopes of these plots are calculated in MATLAB we find:

Observed orders: $L_{\text{inf}} \sim h^{2.019}$, $L2 \sim h^{2.065}$, $L1 \sim h^{2.019}$

As expected, each of the orders is around 2, which indicates an agreement with our truncation error.

Finally, we compare the results of our better scheme. To measure how much better the new scheme is, we will look at number of iterations required for convergence, error, and how smooth the contour plots are (if there is any difference). Figures 4 and 5 below depict these results.

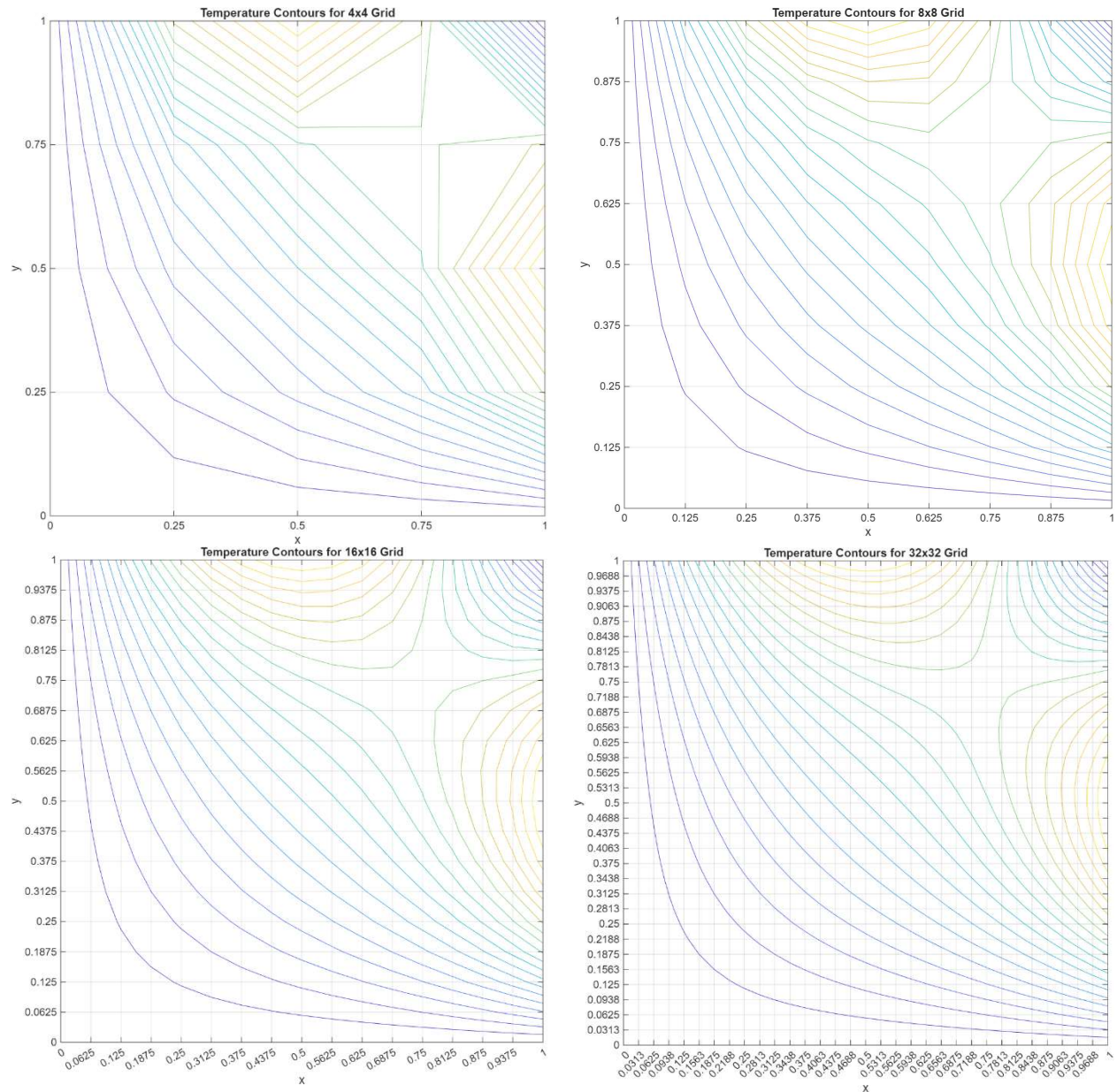


Figure. 4. Updated Contour Plots for “Better” Scheme

It appears that the progression of the very clearly segmented results to the smoother results remains the same, however, the better scheme does actually seem to do a better job of narrowing down the behavior in the center of the contours much sooner. I would say it has done a more reasonable job.

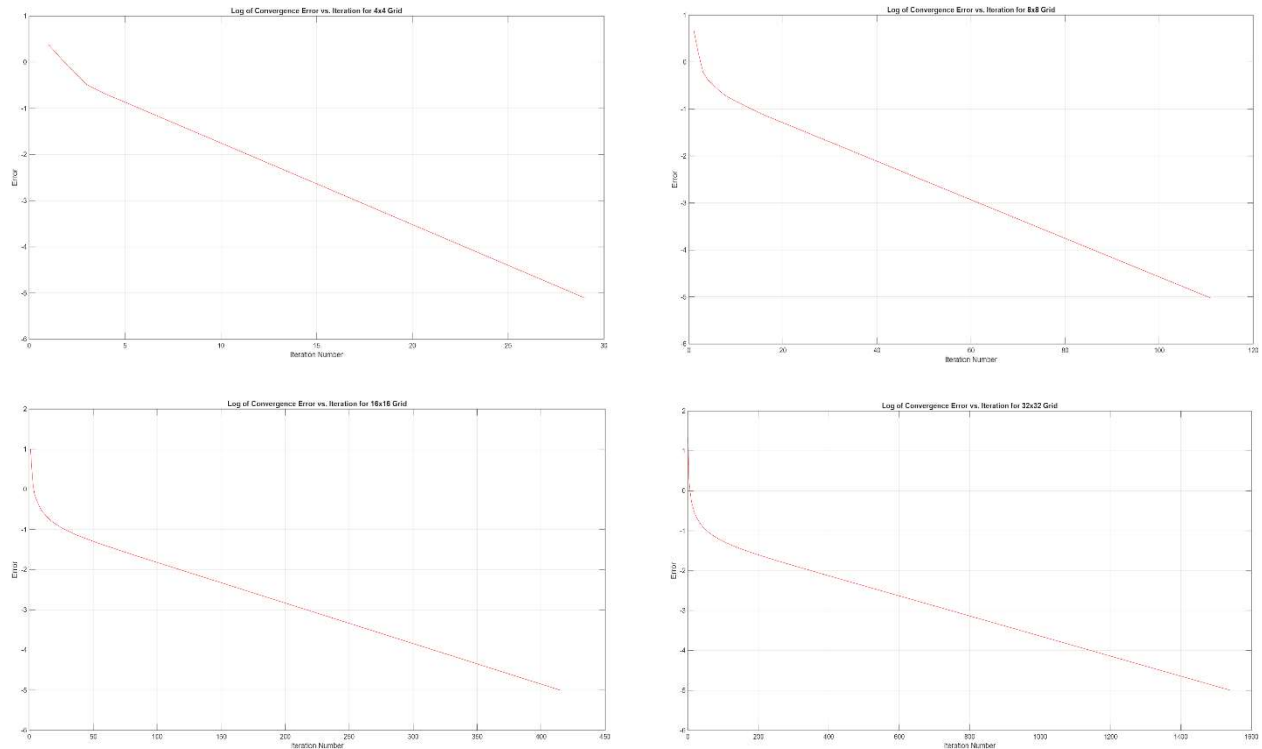


Figure. 5. Convergence history plots of the better scheme.

The general behavior of convergence appears largely the same between the two schemes, however, when checking the number of iterations required for convergence, we do see a slight decrease between the given and better schemes. The better scheme does an okay job of reducing the time required for convergence, which I would say successfully marks our scheme as better. For example, the 32x32 grid requires ~1800 iterations for the given scheme, but our better scheme only takes ~1550 iterations. Not a huge difference, but an improvement, nonetheless.

Conclusion –

This project compared finite-difference solution methods to an exact solution for a 2D temperature plate in thermal equilibrium. It used grid sizes 4x4, 8x8, 16x16, and 32x32 to observe behavioral trends in accuracy and convergence. We also compared errors across solutions and even tested out a better scheme. It was surprising how close such a simple scheme was to the exact, continuous solution.

Eleni Kambouris CFD Project 1

Mesh Grid Simulation

```
clc
clear
close all

% Set Up Grid Size (Square)
gridSize = [4 8 16 32];
N = gridSize + 1;
h = 1./gridSize;
exactFiniteErrorInf = zeros(1,4);
exactFiniteErrorL2 = zeros(1,4);
exactFiniteErrorL1 = zeros(1,4);

% Set Up Initial Loop to Iterate Through All 4 Grid Sizes
for g = 1:1:length(gridSize)

    % Preallocate Matrices (and by default set initial conditions = 0;)
    T = zeros(N(g)); % Iterative Solution
    T_n = zeros(N(g));

    % Set Up Nodes
    x = 0:h(g):1;
    y = 0:h(g):1;
    [X,Y] = meshgrid(x,y);

    % Compute the Exact Solution -
    T_Exact = zeros(N(g),N(g));
    for m = 1:length(x)
        for n = 1:length(y)
            T_Exact(m,n) = 1/sinh(pi)*(sinh(pi*x(m))*sin(pi*y(n))
+sinh(pi*y(n))*sin(pi*x(m)));
        end
    end

    % Return to Iterative Solution
    % Boundary Conditions
    T(1,:) = 0;
    T(:,1) = 0;
    T(end,:) = sin(pi*y);
    T(:,end) = sin(pi*x);

    % Convergence Criteria
    eps = 1*10^-5;
    infErr = 1;
    infError = [];
    l2Error = [];
    l1Error = [];

    while infErr > eps
```

```

    for j = 2:gridSize(g)
        for k = 2:gridSize(g)

            T(j,k) = 0.25 * (T_n(j-1,k) + T_n(j+1,k) + T_n(j,k-1) +
T_n(j,k+1)); % Laplace Scheme
            %T(j,k) = (1/20)*(4 * (T_n(j-1,k) + T_n(j+1,k) + T_n(j,k-1)
+ T_n(j,k+1)))+(T_n(j+1,k+1)+T_n(j+1,k-1)+T_n(j-1,k+1)+T_n(j-1,k-1))); %
Better Scheme (I Hope)
        end

    end

    % Calc Error Through L_inf
    infErr = norm((T_n-T),inf);
    infError = [infError; infErr];

    % L2,L1 Errors
    l2Err = norm((T_n-T),2);
    l2Error = [l2Error;l2Err];

    l1Err = norm((T_n-T),1);
    l1Error = [l1Error;l1Err];

    diff = T_Exact - T;
    exactFiniteErrorInf(g) = norm(diff,inf);
    exactFiniteErrorL2(g) = norm(diff,2);
    exactFiniteErrorL1(g) = norm(diff,1);

    % Update Matrix
    T_n = T;

end

% Contour Plot
figure;
contour(X, Y, T_n, 0:0.05:1)
title(sprintf('Temperature Contours for %dx%d Grid', gridSize(g),
gridSize(g)))
xlabel('x')
ylabel('y')
xticks(x)
yticks(y)
axis equal tight
grid on
hold off

% Plot Convergence History
figure;
ax = 1:1:length(infError);
plot(ax,log10(infError),'-r')
title(sprintf('Log of Convergence Error vs. Iteration for %dx%d Grid',
gridSize(g), gridSize(g)))
xlabel("Iteration Number")
ylabel("Error")
grid on

```

```

    % Uncomment For Running/ Comment for Testing
    % Output Solution Files
    % Temperature Distribution -
    %filename = sprintf('temperatureGrid_dx%d.txt', gridSize(g),
gridSize(g));
    %writematrix(T_n, filename);

    % Convergence History
    %filename = sprintf('convergenceHistory_dx%d.txt', gridSize(g),
gridSize(g));
    %writematrix(error, filename);
end

fprintf("Observed Errors Between Exact and Numerical Solutions (Linf): 4x4 =
%.4f, 8x8 = %.4f, 16x16 = %.4f, 32x32 = %.4f\n ",exactFiniteErrorInf)
fprintf("Observed Errors Between Exact and Numerical Solutions (L2): 4x4 =
%.4f, 8x8 = %.4f, 16x16 = %.4f, 32x32 = %.4f\n ",exactFiniteErrorL2)
fprintf("Observed Errors Between Exact and Numerical Solutions (L1): 4x4 =
%.4f, 8x8 = %.4f, 16x16 = %.4f, 32x32 = %.4f\n ",exactFiniteErrorL1)

% Plot Solution Error Vs. Mesh Size (h) For Each Norm
figure;
plot(log10(h),log10(exactFiniteErrorL1))
title("LogLog Of Solution Error vs. Mesh Size (h) For L1 Norm")
xlabel("h")
ylabel("Solution Error")
grid on
hold off

figure;
plot(log10(h),log10(exactFiniteErrorL2))
title("LogLog Of Solution Error vs. Mesh Size (h) For L2 Norm")
xlabel("h")
ylabel("Solution Error")
grid on
hold off

figure;
plot(log10(h),log10(exactFiniteErrorInf))
title("LogLog Of Solution Error vs. Mesh Size (h) For Inf Norm")
xlabel("h")
ylabel("Solution Error")
grid on
hold off

% Compute Slopes
p_inf = polyfit(log10(h), log10(exactFiniteErrorInf.*h), 1); pLinf =
p_inf(1);
p_L2 = polyfit(log10(h), log10(exactFiniteErrorL2.*h), 1); pL2 = p_L2(1);
p_L1 = polyfit(log10(h), log10(exactFiniteErrorL1.*h), 1); pL1 = p_L1(1);
fprintf('Observed orders: L_inf ~ h^{%.3f}, L2 ~ h^{%.3f}, L1 ~ h^{%.3f}\n',
pLinf, pL2, pL1);

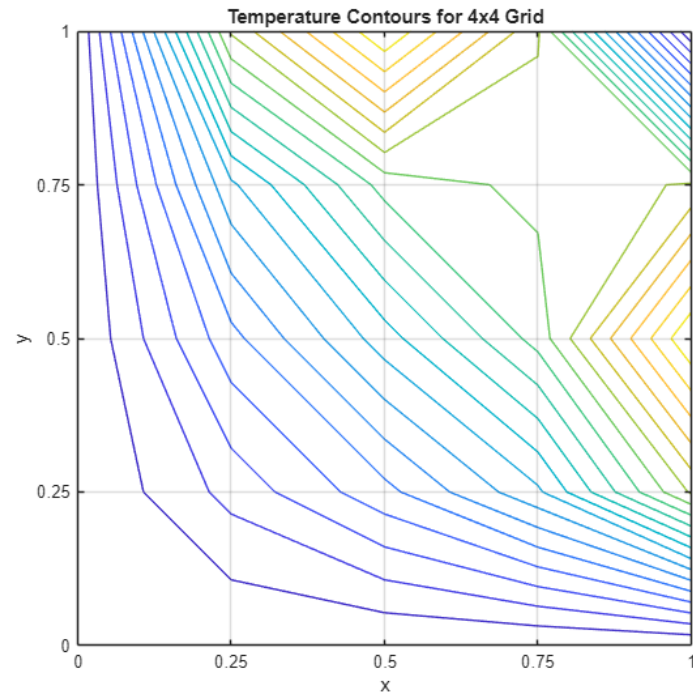
```

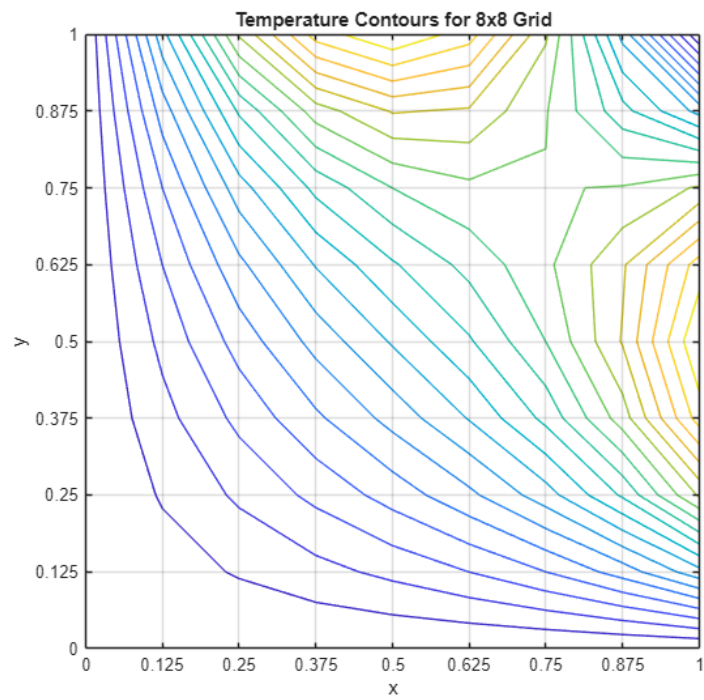
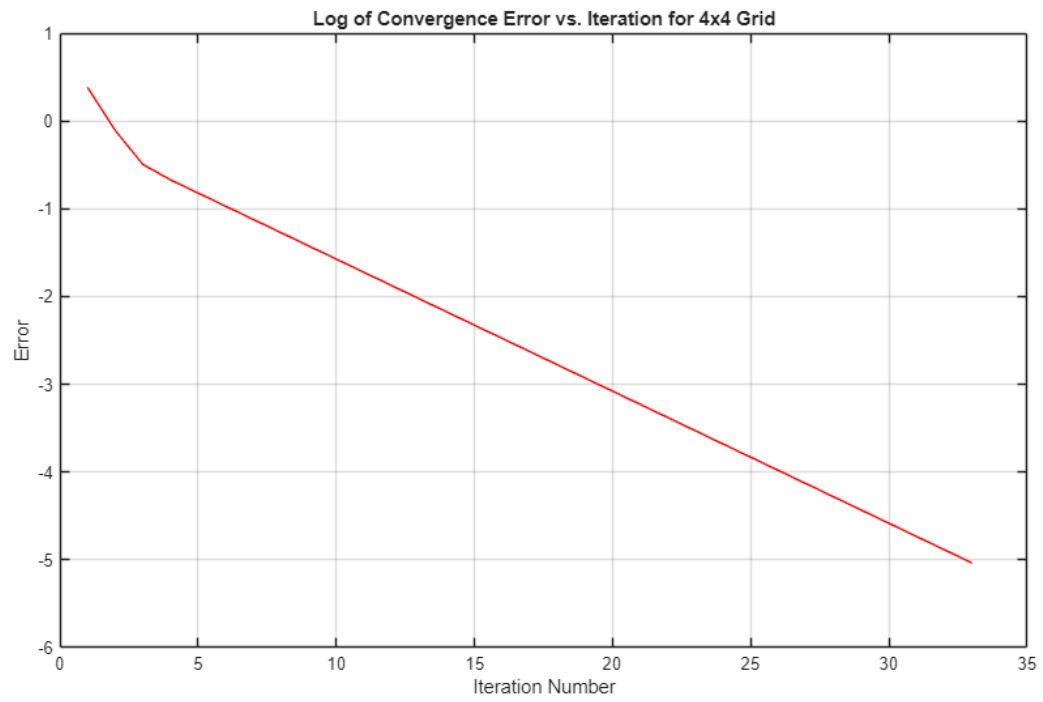
Observed Errors Between Exact and Numerical Solutions (L_{∞}): $4 \times 4 = 0.0721$,
 $8 \times 8 = 0.0408$, $16 \times 16 = 0.0205$, $32 \times 32 = 0.0086$

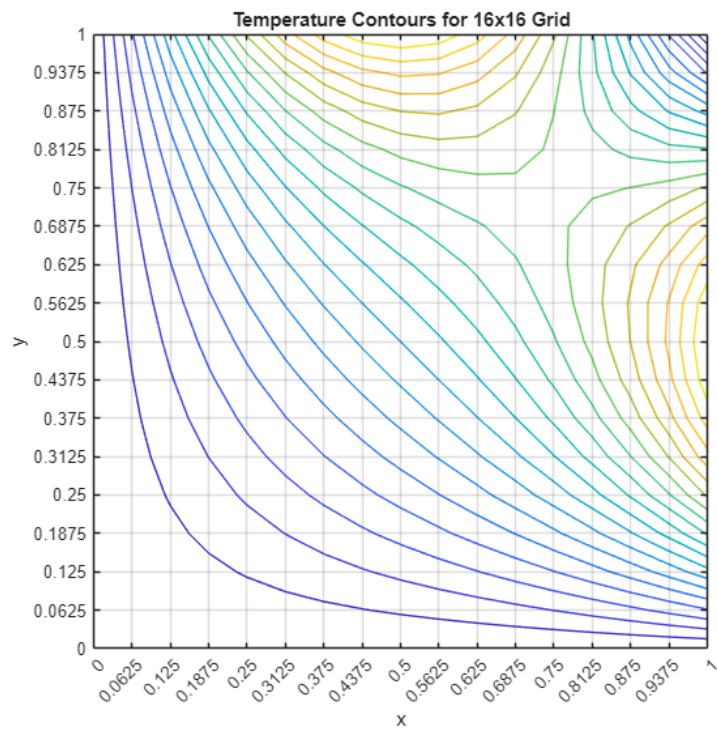
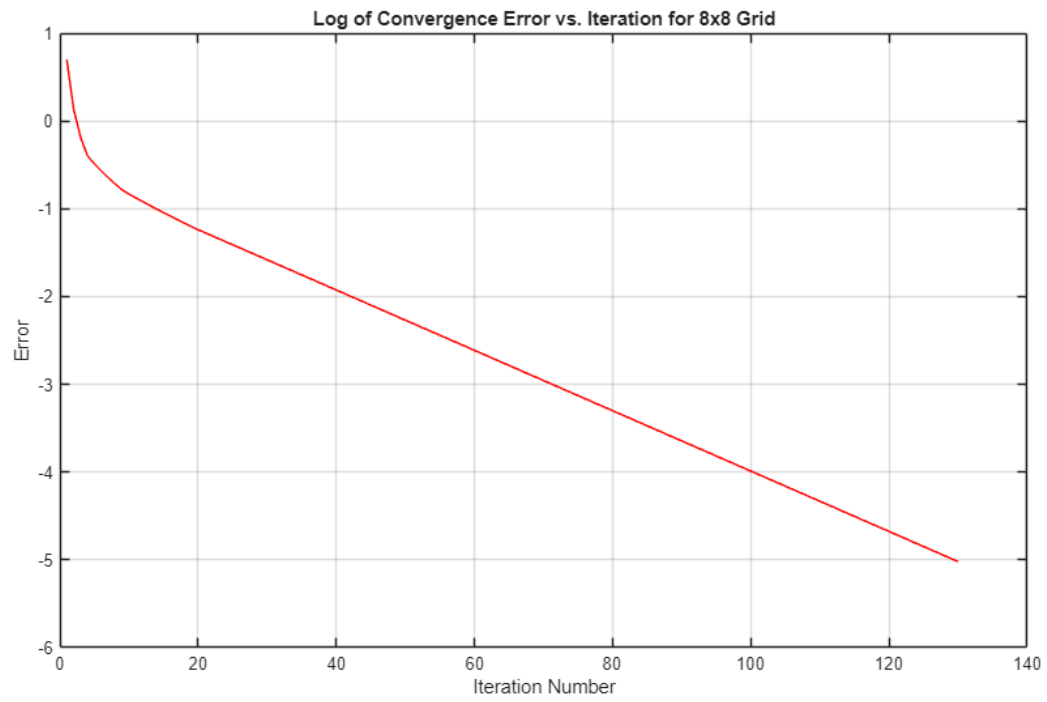
Observed Errors Between Exact and Numerical Solutions (L_2): $4 \times 4 = 0.0635$,
 $8 \times 8 = 0.0331$, $16 \times 16 = 0.0164$, $32 \times 32 = 0.0069$

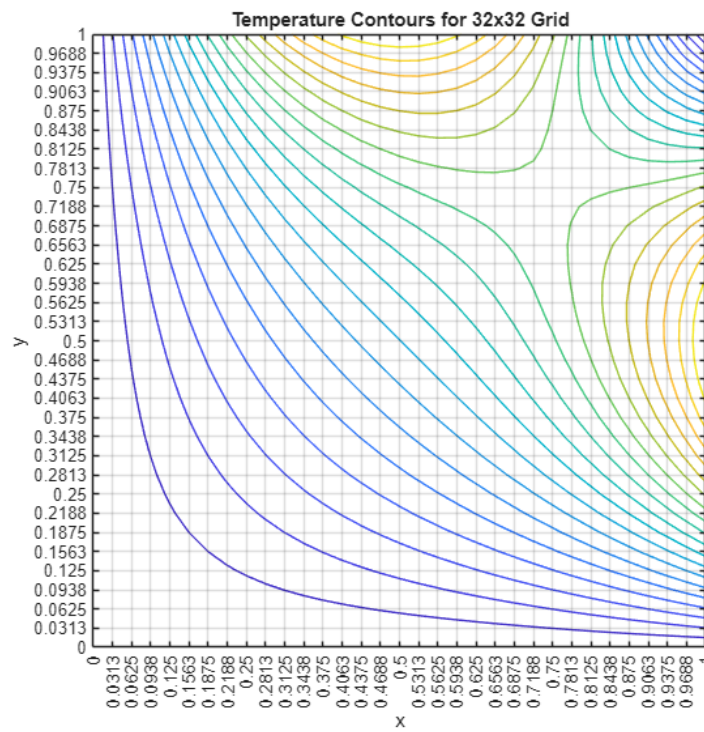
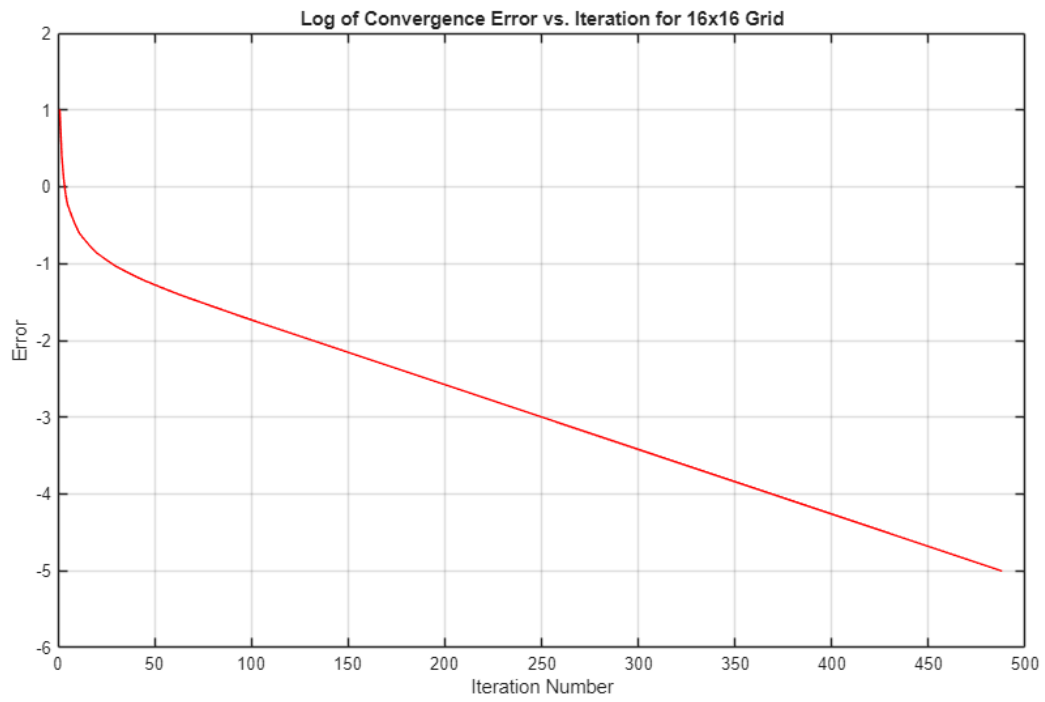
Observed Errors Between Exact and Numerical Solutions (L_1): $4 \times 4 = 0.0721$,
 $8 \times 8 = 0.0408$, $16 \times 16 = 0.0205$, $32 \times 32 = 0.0086$

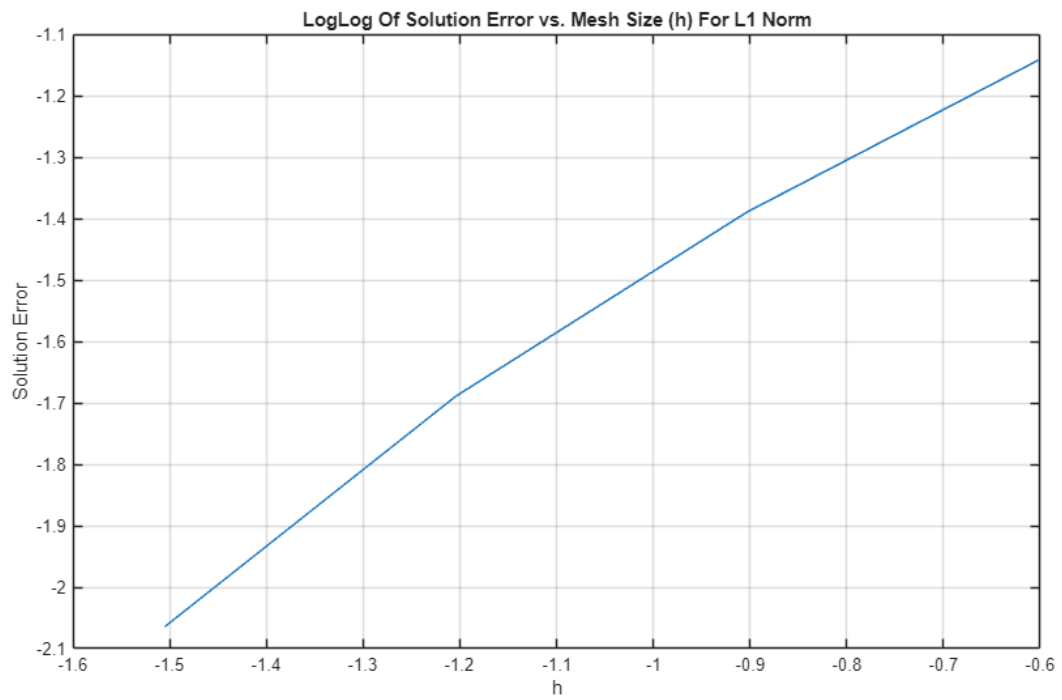
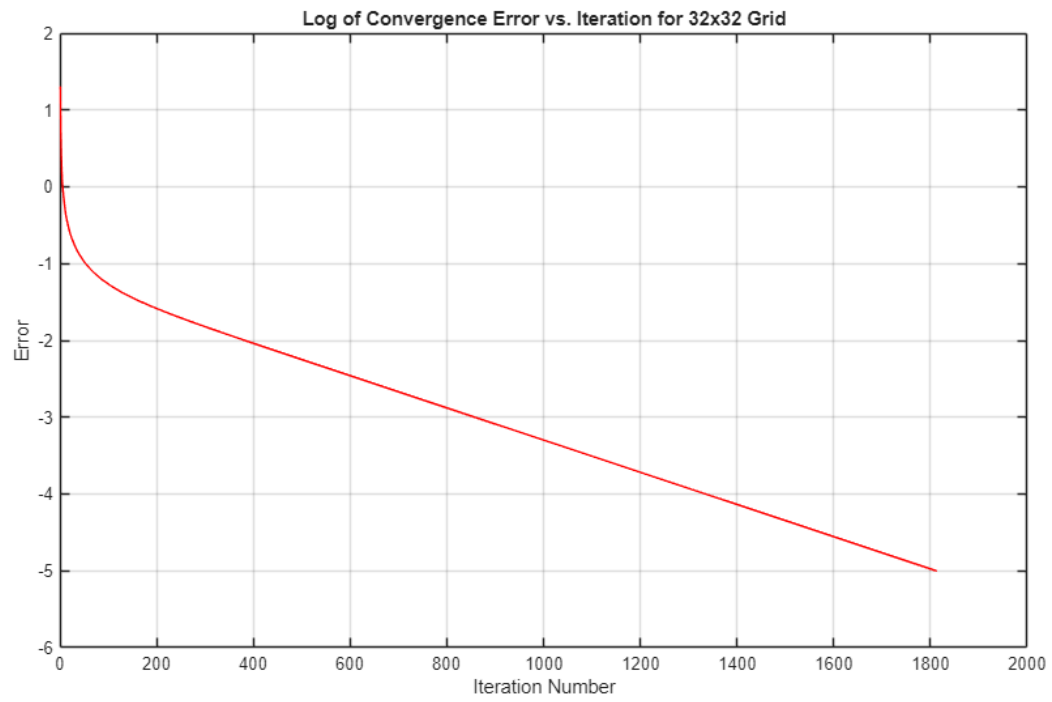
Observed orders: $L_{\infty} \sim h^{2.019}$, $L_2 \sim h^{2.065}$, $L_1 \sim h^{2.019}$

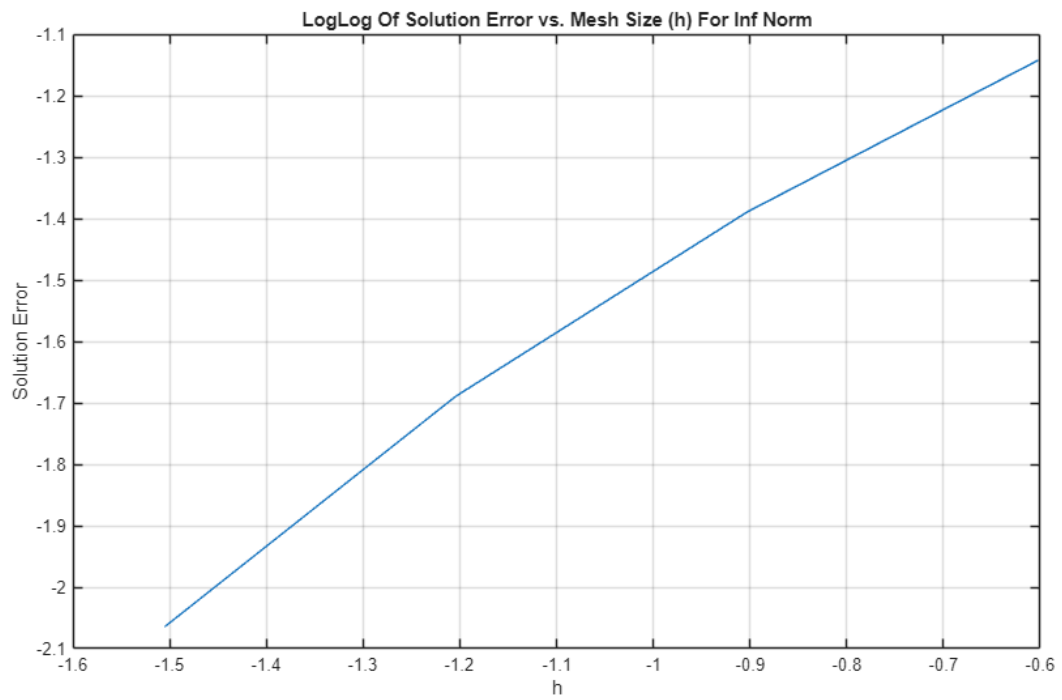
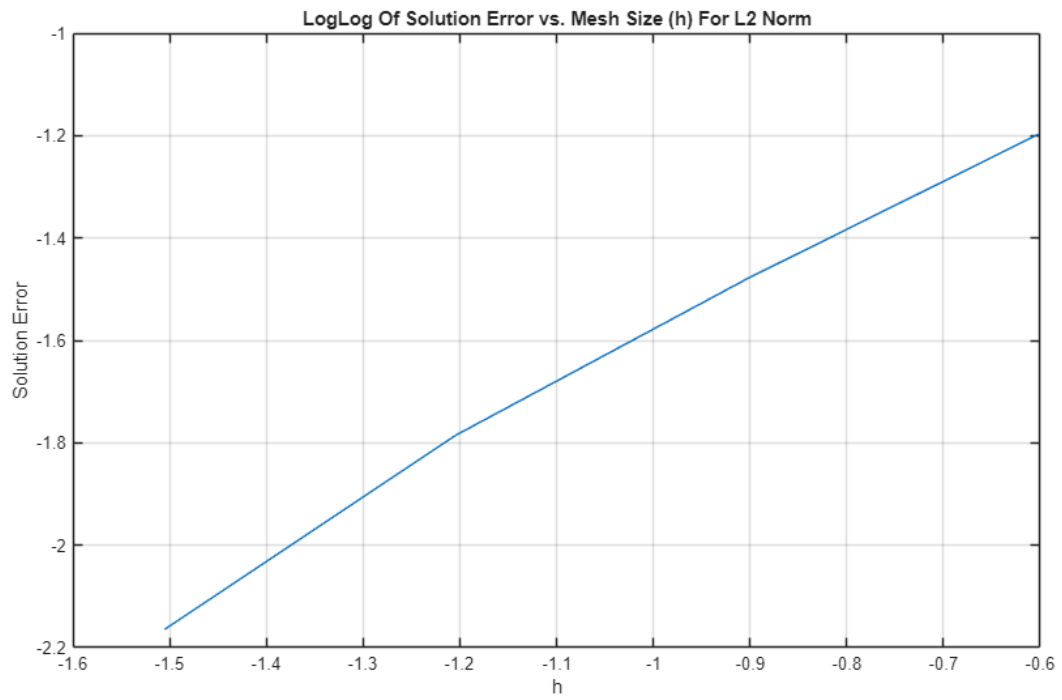












Appendix B DERIVE TRUNCATION ERROR (STEADY-STATE: $T^{n+1} \equiv T^n$):

$$\hookrightarrow T_{j,k} = \frac{1}{4} (T_{j-1,k} + T_{j+1,k} + T_{j,k-1} + T_{j,k+1})$$

AND WE HAVE $\Delta x = \Delta y = h$ FOR j, k

$$(2) \left[T_{j,k} = \frac{1}{4} (T_{j-1,k} + T_{j+1,k} + T_{j,k-1} + T_{j,k+1}) \right]$$

$$\hookrightarrow 2 \cdot T_{j,k} = \frac{1}{2} (T_{j-1,k} + T_{j+1,k}) + \frac{1}{2} (T_{j,k+1} + T_{j,k-1})$$

$$\frac{1}{h^2} (0 = \frac{1}{2} (T_{j-1,k} + T_{j+1,k}) + \frac{1}{2} (T_{j,k+1} + T_{j,k-1}) - 2T_{j,k})$$

$$O(h^2) + 0 = \frac{\frac{1}{2} (T_{j-1,k} + T_{j+1,k}) - T_{j,k}}{h^2} + \frac{\frac{1}{2} (T_{j,k+1} + T_{j,k-1}) - T_{j,k}}{h^2}$$

↑ technically, b/c T.E. bounded by $\propto h^2$

$j \hat{=} x, k \hat{=} y$

$$\hookrightarrow T_{j \pm 1, k} = T_{j,k} \pm \Delta x \frac{\partial T}{\partial x} + \frac{\Delta x^2}{2} \frac{\partial^2 T}{\partial x^2} \pm \frac{\Delta x^3}{6} \frac{\partial^3 T}{\partial x^3} + \frac{\Delta x^4}{24} \frac{\partial^4 T}{\partial x^4} \pm \frac{\Delta x^5}{120} \frac{\partial^5 T}{\partial x^5} + O(\Delta x^6)$$

$$\begin{aligned} \hookrightarrow T_{j+1,k} + T_{j-1,k} &= \left(T_{j,k} + \cancel{\Delta x \frac{\partial T}{\partial x}} + \frac{\Delta x^2}{2} \frac{\partial^2 T}{\partial x^2} + \cancel{\frac{\Delta x^3}{6} \frac{\partial^3 T}{\partial x^3}} + \frac{\Delta x^4}{24} \frac{\partial^4 T}{\partial x^4} + \cancel{\frac{\Delta x^5}{120} \frac{\partial^5 T}{\partial x^5}} + O(\Delta x^6) \right) \\ &+ \left(T_{j,k} - \cancel{\Delta x \frac{\partial T}{\partial x}} + \frac{\Delta x^2}{2} \frac{\partial^2 T}{\partial x^2} - \cancel{\frac{\Delta x^3}{6} \frac{\partial^3 T}{\partial x^3}} + \frac{\Delta x^4}{24} \frac{\partial^4 T}{\partial x^4} - \cancel{\frac{\Delta x^5}{120} \frac{\partial^5 T}{\partial x^5}} + O(\Delta x^6) \right) \\ &= 2 \left(T_{j,k} + \frac{\Delta x^2}{2} \frac{\partial^2 T}{\partial x^2} + \frac{\Delta x^4}{24} \frac{\partial^4 T}{\partial x^4} + O(\Delta x^6) \right) \end{aligned}$$

$$\hookrightarrow \frac{1}{2} (T_{j+1,k} + T_{j-1,k}) = T_{j,k} + \frac{\Delta x^2}{2} \frac{\partial^2 T}{\partial x^2} + \frac{\Delta x^4}{24} \frac{\partial^4 T}{\partial x^4} + O(\Delta x^6)$$

AND ANALOGOUSLY, WE HAVE -

$$\frac{1}{2} (T_{j,k+1} + T_{j,k-1}) = T_{j,k} + \frac{\Delta y^2}{2} \frac{\partial^2 T}{\partial y^2} + \frac{\Delta y^4}{24} \frac{\partial^4 T}{\partial y^4} + O(\Delta y^6)$$

$\hookrightarrow$ PLUG BACK IN - AND $h^2 = \Delta x^2 = \Delta y^2$

$$= \frac{1}{2} \frac{\partial^2 T}{\partial x^2} + \frac{h^2}{24} \frac{\partial^4 T}{\partial x^4} + \frac{1}{2} \frac{\partial^2 T}{\partial y^2} + \frac{h^2}{24} \frac{\partial^4 T}{\partial y^4} + O(h^4)$$

$$= \frac{1}{2} \left(\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} \right) + \frac{h^2}{24} \left(\frac{\partial^4 T}{\partial x^4} + \frac{\partial^4 T}{\partial y^4} \right) + O(h^4)$$

$$\text{T.E.} = \rho_h \mathcal{L}(u) - \mathcal{L}_h(\rho_h u_h) = \nabla^2 T - \nabla^2 T + \frac{h^2}{24} \left(\frac{\partial^4 T}{\partial x^4} + \frac{\partial^4 T}{\partial y^4} \right) + O(h^4)$$

$$\tau = \frac{h^2}{24} \left(\frac{\partial^4 T}{\partial x^4} + \frac{\partial^4 T}{\partial y^4} \right) + O(h^4)$$

THE LOWEST TERM IS PROPORTIONAL TO $h^2 \rightarrow$ 2nd-ORDER ACCURATE. $\tau \sim O(h^2)$

CONSISTENCY OF THE SCHEME -

$$\tau = 0 \quad \text{AS } h \rightarrow 0$$

FOR $h=0 \rightarrow \tau=0$ ✓ THE SCHEME IS CONSISTENT!

DERIVE THE EXACT SOLUTION - SEPARATION OF VARIABLES

$$\nabla^2 T = 0$$

$$\text{w/ } T(x, y) = F(x)G(y)$$

AND

$$T(0, y) = 0$$

$$T(x, 0) = 0$$

$$T(1, y) = \sin(\pi y)$$

$$T(x, 1) = \sin(\pi x)$$

$$\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} = 0$$

↳ w/ $T(x, y) = F(x)G(y)$ GET -

$$F_{xx}G + FG_{yy} = 0$$

$$\hookrightarrow \frac{F_{xx}}{F} = -\frac{G_{yy}}{G} = \lambda \quad \rightarrow \quad f_{xx} - \lambda f = 0 \quad \text{; } \quad g_{yy} + \lambda g = 0$$

BUT, WE HAVE 2 NON-HOMOGENEOUS B.C.s, SO USE SUPERPOSITION

$$\hookrightarrow T(x, y) = T_1(x, y) + T_2(x, y)$$

$$\text{START w/ } T_1(x, y) \rightsquigarrow \text{BCs: } T_1(0, y) = 0, T_1(x, 0) = 0,$$

$$T_1(x, 1) = 0, T_1(1, y) = \sin(\pi y)$$

$$\hookrightarrow \text{solving } G(y) \rightsquigarrow \lambda_n = (n\pi)^2 \quad n = 1, 2, 3, \dots$$

$$G_n(y) = \sin(n\pi y)$$

YIELDS

$$F_{xx,n} - (n\pi)^2 F_n = 0$$

$$f_n(x) = A_n \sinh(n\pi x) + B_n \cosh(n\pi x)$$

$$F(0) = 0 = B_n$$

$$\leadsto F_n(x) = A_n \sinh(n\pi x)$$

$$\hookrightarrow T_1(x, y) = A_n \sinh(n\pi x) \sin(n\pi y)$$

$$w) T_1(x, y) = \sum_{n=1}^{\infty} A_n \sinh(n\pi x) \sin(n\pi y)$$

$$T(1, y) = \sum_{n=1}^{\infty} A_n \sinh(n\pi) \sin(n\pi y) = \sin(\pi y) \\ = \sum_{n=1}^{\infty} c_n \sin(n\pi y), \quad c_1 = \int_0^1 (1 - \cos(\pi y)) dy = 1$$

$$\hookrightarrow T_1(x, y) = \frac{\sinh(\pi x)}{\sinh(\pi)} \cdot \sin(\pi y)$$

Now for $T_2(x, y)$, from symmetry -

$$T_2(x, y) \leadsto \text{BCs: } T(0, y) = 0, T(x, 0) = 0 \\ T(1, y) = 0, T(x, 1) = \sin(\pi x)$$

$$T_2(x, y) = \frac{\sinh(\pi y)}{\sinh(\pi)} \cdot \sin(\pi x)$$

$$\leadsto \boxed{T(x, y) = \frac{\sinh(\pi x)}{\sinh(\pi)} \cdot \sin(\pi y) + \frac{\sinh(\pi y)}{\sinh(\pi)} \cdot \sin(\pi x)}$$